



59 / 59

Backend Tests Passing

83.4%

Asset Payload Reduction

100%

Verified Student Gating

₹0

Platform Deduction (UPI)

1. Executive Summary & Problem Context

CampusRide is an enterprise-grade university carpooling and student mobility network built to address the critical transport bottleneck experienced daily by university students across Dehradun, Uttarakhand (Uttaranchal University, UPES, Graphic Era).

Prior to CampusRide, students relied on informal WhatsApp groups with unverified strangers, unsafe vehicle overloading, predatory auto-rickshaw fares, and zero digital safety oversight. CampusRide solves this with a verified academic trust framework: strict institutional email and ID card validation, real-time road corridor matching, daily driver liveness checks, and instant WhatsApp emergency dispatch.

2. Architectural & Technical Stack

Layer	Technology	Implementation Details
Frontend Architecture	React 18, TypeScript, Vite, Tailwind CSS	Fully typed SPAs, component-driven modular structure, responsive mobile-first UI.
Animation & Motion	Framer Motion v13 & GSAP 3.12	Physics-based modals, confetti feedback, animated SVG checkmarks, and page transitions.
Mapping & Navigation	OSRM Road Engine, Leaflet, CARTO Voyager	Polyline route rendering, multi-stop corridor snapping, and campus gate markers.
Backend API Server	Node.js, Express, TypeScript	RESTful micro-routes, Zod schema validation, Helmet CSP, Express Rate Limit, HPP.
Database Engine	MongoDB & Mongoose (Atlas / Embedded)	Geospatial 2dsphere indexes, compound indexes, atomic seat decrement transactions.
Real-Time Communications	Socket.IO & Baileys WhatsApp Engine	Bidirectional chat synchronization, live driver GPS coordinates, and WhatsApp OTP alerts.
Client Mobile Experience	Progressive Web App (PWA)	W3C Web Manifest, Service Worker offline caching, home-screen installation.

3. High-Impact Daily Commute Upgrades

To eliminate friction from repetitive manual actions and ensure high student driver retention, the following daily commute features were designed and deployed:

- Recurring Weekday Commutes (Mon–Fri Schedules):** Drivers can toggle a semester-long recurring schedule. The ride engine automatically surfaces the ride for regular morning slots (e.g. 08:30 AM),

eliminating the need to post manually every day. Handled through embedded Mongoose schema subdocuments (`daysOfWeek: [1,2,3,4,5]` , `time: string`).

- **Zero-Commission Direct UPI Fare Settlement:** Drivers store their UPI VPA (e.g., `driver@okhdfcbank`). When a trip finishes, passengers receive a direct settlement modal with standard `upi://pay?pa=...&am=...` deep links for Google Pay, PhonePe, and Paytm, accompanied by an instant copy-to-clipboard action. Zero payment gateway commissions are deducted.
- **Quick Commute Chat Presets:** Pre-programmed one-tap coordination chips embedded in the in-ride coordination chat modal (" *I've arrived at pickup gate*", " *Running 3 mins late*", " *Waiting by library steps*") allow instantaneous communication without typing while walking or driving.
- **Campus Landmark Selection:** Rapid one-tap selector pills for all major campus hubs: UIT Engineering Porch, USCS Computing Quad, Central Library, Premnagar Chowk Market, and Suddhowala PG hub.



4. Trust, Safety & Verification Architecture

Zero-Trust Academic Verification Gating:

Unverified or pending accounts are strictly barred on both frontend UI layers and backend server endpoints. Unverified students cannot create rides (`403 FORBIDDEN`) or send seat requests (`VERIFICATION_REQUIRED`).

A. Client-Side Student ID Card OCR Pre-Screening

An intelligent client-side OCR pre-screening parser was integrated directly into the verification submission workflow. When a student uploads their college ID card image, the system inspects the image buffer, automatically extracting student roll numbers, college names, and academic branches, pre-populating fields and accelerating administrative review.

B. Live WhatsApp Emergency SOS Integration

During an active trip, passengers and drivers have access to a one-tap SOS safety system. Triggering the emergency action automatically records an `EmergencyIncident` in MongoDB, idempotently prevents duplicated alarms, and dispatches a pre-formatted WhatsApp alert containing real-time Google Maps GPS coordinates directly to the user's designated emergency contacts.

C. Daily Physical Driver ID Check

Before starting the first trip of each day, drivers must complete a physical student ID verification step via `DailyDriverIdCheckModal`. This ensures the driver operating the vehicle matches the registered student credentials on file.

D. Verified Driver Badges & Academic Affinity

Ride cards now render visual trust badges so passengers can make informed decisions before requesting:

 Classmate / Same Dept Top Punctual Driver (10+ rides) Eco Commuter (50+ kg CO₂ saved) Mon-Fri Regular Commute

5. Asset Optimization & Performance (WebP Conversion)

Initial audits revealed that full-screen illustrated backgrounds in PNG and high-resolution JPEG formats consumed over **25.80 MB** in aggregate, causing noticeable load latency on 4G/5G mobile connections.

Using an automated pipeline powered by `sharp`, all 33 illustration assets were converted to modern WebP with smart subsampling and high-fidelity compression (quality 82, effort 6).

Illustration Asset	Original Format & Size	WebP Size	Bandwidth Reduction
bg-auth.webp	PNG — 1,977 KB	136 KB	-93.1%
safety-scrapbook.webp	PNG — 451 KB	35 KB	-92.3%
admin-analytics-bg.webp	PNG — 737 KB	95 KB	-87.1%
bg-home.webp	PNG — 685 KB	98 KB	-85.8%
bg-dashboard.webp	PNG — 759 KB	109 KB	-85.6%
verification-hero.webp	JPG — 659 KB	100 KB	-84.8%
campus-reviews-trust.webp	JPG — 808 KB	125 KB	-84.6%
find-ride.webp	JPG — 798 KB	154 KB	-80.8%
offer-ride.webp	JPG — 815 KB	162 KB	-80.1%
auth-student.webp	JPG — 1,037 KB	209 KB	-79.8%
Total Across All 33 Assets	25.80 MB	4.28 MB	83.4% TOTAL REDUCTION

6. Progressive Web App (PWA) Integration

CampusRide is now an installable PWA on Android, iOS, and desktop browsers:

- **Web App Manifest:** Configured at `client/public/manifest.json` with dark forest green theme (`#143D32`), standalone display mode, and high-resolution icons.
- **Service Worker:** Implemented at `client/public/sw.js` with cache-first static asset delivery, network-first API dispatch, and full offline resilience. Registered in `main.tsx` .



7. Critical Bug Fixes & Code Hardening

Component / Module	Root Cause	Surgical Resolution
Recurring Commute Mongoose Schema	<code>recurringSchedule</code> was passed as a string or unstructured object, throwing Mongoose cast error <code>ObjectExpectedError</code> on ride creation.	Added <code>recurring: { type: Boolean, default: false }</code> to <code>RideSchema</code> and sanitized <code>recurringSchedule</code> into structured <code>{ daysOfWeek: [1,2,3,4,5], time }</code> in <code>rideRoutes.ts</code> .
False-Positive Driver Check-In for Passengers	In <code>RideDetailPage.tsx</code> and <code>TripTrackingPage.tsx</code> , <code>isDriver</code> was computed as <code>ride?.creator?._id === user?._id</code> . On initial load, both were <code>undefined</code> , causing <code>undefined === undefined</code> to evaluate to <code>true</code> and flash driver actions to passengers.	Hardened with safe ID extraction and boolean validation: <code>Boolean(rideCreatorId && currentUserId && rideCreatorId === currentUserId)</code> .
Stale Request Cache on Unverified Users	Search page read stale <code>campusrider_local_requests</code> from <code>localStorage</code> during initial render state initialization, briefly showing "Request Sent" before verification sync.	Added verification check in initial <code>useState</code> initializer: if user is not verified, returns an empty set immediately.

8. Quality Assurance & Test Suite Results

The backend test suite runs with Jest against isolated in-memory MongoDB instances, validating core security guardrails, role boundaries, cryptographic CSPRNG OTP generation, and end-to-end user workflows:

Test Suite	Scope & Guardrails Verified	Result
<code>securityPatch.test.ts</code>	Guardrail #1-4: Seed endpoint admin keys, PII sanitization, compound unique request indexes, atomic seat decrement, CSPRNG OTP generation & 5-attempt logout.	PASS (10 TESTS)
<code>e2eVerification.test.ts</code>	Full 10-step E2E flow: Healthcheck, driver/passenger login, ride offer, matching score breakdown, seat request/accept, real-time chat, OTP trip completion, ratings, admin analytics.	PASS (10 TESTS)
<code>securityAndIntegrations.test.ts</code>	RBAC self-verification prevention, admin audit log protection, SOS incident idempotency, unauthorized OTP rejection, invalid trip state machine transition rejection.	PASS (11 TESTS)
<code>realUserRegistrationAndBiometrics.test.ts</code>	Registration with custom academic profiles, profile updates, driver avatar biometrics, email dispatch simulation.	PASS (4 TESTS)
<code>authorizationBypass.test.ts</code>	Broken Object Level Auth (BOLA): User B blocked from deleting or accepting requests on User A's rides; missing or tampered JWT signatures rejected.	PASS (7 TESTS)
<code>matchingEngine.test.ts</code>	Matching algorithm accuracy, directional alignment, seat availability filter, time window clamping, detour calculations.	PASS (6 TESTS)
<code>corridorDirectionAndQA.test.ts</code>	Reverse route disqualification, campus hub proximity boundaries, academic affinity hierarchy (Classmate Rank 1, Department Rank 2), women-only driver gating.	PASS (11 TESTS)
Total Across 7 Suites	Comprehensive End-to-End & Security Regression Coverage	59 / 59 PASSED (100%)

9. Production Build & Deployment Status

Build Status: Both `npm --prefix client run build` and `npm --prefix server run build` compile with **0 TypeScript errors**. Production assets are minified, gzip-optimized, and deployed to `origin/main` at commit `9513df7`.